RUHU ARCHITECTURE REFERENCE

Production Conversational AI Runtime

Ruhu is a turn-based conversational AI platform built around an authored `AgentDocument`. The backend kernel executes scenarios, steps, guards, transitions, tools, facts, rules, response generation, voice sessions, public widgets, ticketing, analytics, and Atlas operational automation.

Current architecture stance. The runtime underneath is general-purpose: the authored agent document, tool runtime, workflow/journey modules, knowledge retrieval, ticketing, public widget, telephony, and Atlas automation are general-purpose. The product can still package region-specific carriers, payments, compliance, and support workflows as deployment packs on top of the same workflow-agent platform.

Python 3.12+ · FastAPI · SQLAlchemy 2 · Pydantic 2.12 · React 18 · TypeScript 5.9 · Vite 8 · LiveKit Agents 1.5.2

A3 portrait

PDF page format retained from the prior architecture document.

91

Alembic migration files in the current repository.

7

System templates shipped under `src/ruhu/templates/system`.

Atlas

Operational review, readiness, documentation parsing, provisioning, and apply flow.

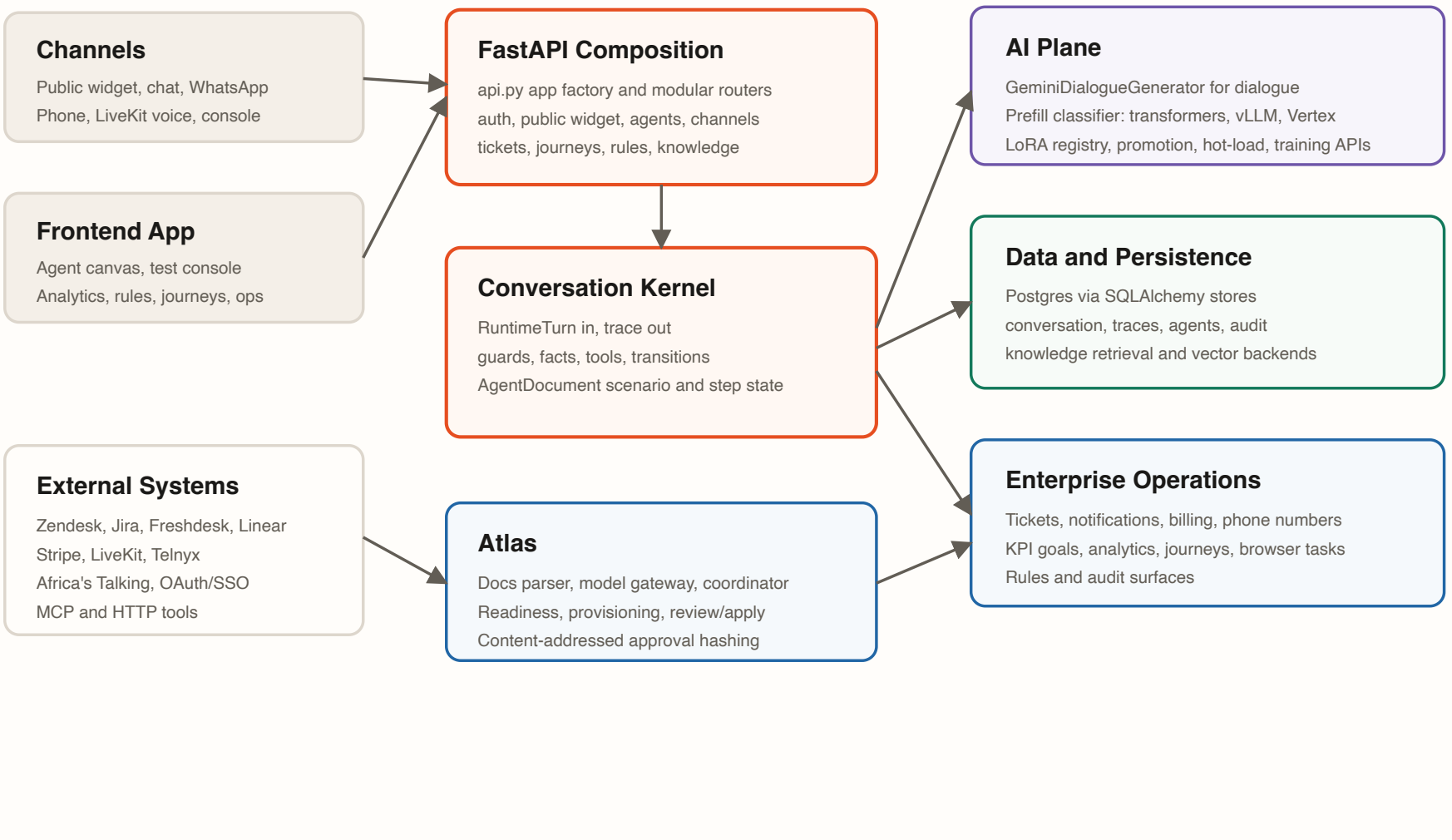
Revision
11 June 2026

Source
Local repository audit

Print setup
A3 portrait, 14mm top/bottom, 18mm left/right

Scope
Backend, frontend, data, AI, Atlas, deployment

01 Executive Architecture



High-level runtime shape. The kernel remains the deterministic center; AI, tools, routes, and operational systems are composed around it.

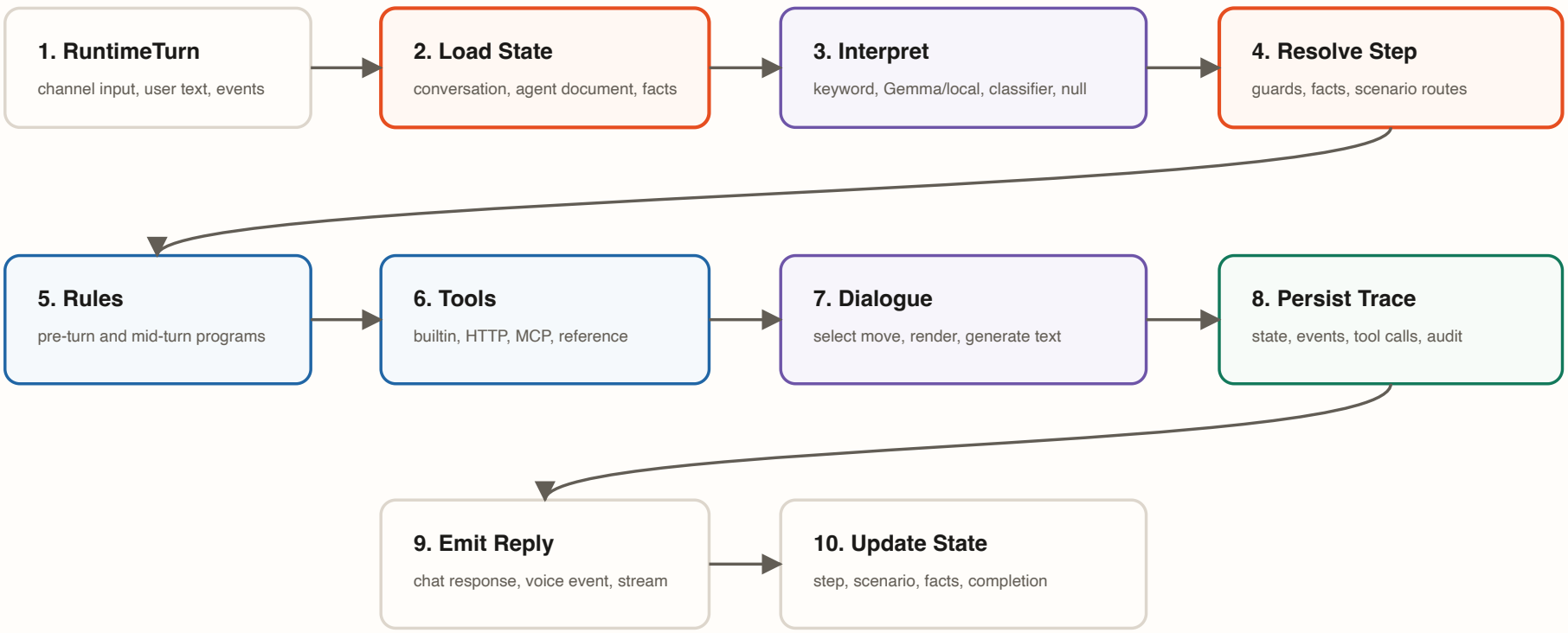
Runtime Contract

- `AgentDocument` remains the authored source of truth for scenarios, steps, transitions, facts, tools, and handoff behavior.
- `ConversationKernel` executes every simulated or live turn through deterministic state loading, guard evaluation, tool policy, and trace persistence.
- LLM components support interpretation, rendering, evaluation, and repair planning; they do not replace runtime state or validation.

Enterprise Control Points

- Versioned agent documents, publish review, audit records, trace storage, and readiness reports support regulated deployment workflows.
- Tool execution goes through controlled built-in, HTTP, MCP, and reference executors with traceable invocation records.
- Atlas proposes typed changes and readiness reports while human approvals control production behavior changes.

02 Backend Runtime



Turn execution is deterministic around state loading, guards, tool policy, transitions, persistence, and channel-specific output.

Backend Area	Current Modules	Role	Status
Kernel	`kernel.py`, `schemas.py`, `agent_document.py`	Core turn-based state machine for scenarios, steps, guards, transitions, tools, and traces.	**Current**
App composition	`api.py`, `composition.py`, `routes/*`	FastAPI app factory, modular route installation, dependency wiring, startup orchestration.	**Current**
Agents	`agent_*.py`, `routes/agents.py`, templates	Agent document CRUD, drafts, versions, validation, publishing, audit, system templates.	**Current**
Channels	`routes/public_widget.py`, `routes/channels.py`, `livekit_*.py`, phone providers	Widget, chat, WhatsApp, phone, LiveKit voice, streaming events, provider webhooks.	**Current**
Operations	`ticketing_api.py`, `notifications/`, `billing/`, `kpi/`, `journeys/`	Enterprise work surfaces beyond chat: ticketing, KPI goals, billing, notifications, journeys.	**Current**
Knowledge	`knowledge/`	Document ingestion, chunking, embedding, retrieval, and runtime knowledge resolution.	**Current**
Tools	`tools/`	Tool runtime and executors for built-in, HTTP, MCP, and reference tools.	**Current**

API boundary note. Steps and transitions are not separate REST resources. The agent document is the unit of edit through `PUT /agents/{agent_id}/agent-document`.

Deployment note. The competition deployment path should be described through Google Cloud hosting, logging, storage, and provider integrations. The runtime does not depend on a separate workflow orchestrator to execute customer conversations.

03 AgentDocument Contract

Top-Level Model

Field	Meaning
`version`	Document schema/version marker.
`start_scenario_id`	Initial scenario used by the kernel.
`scenarios`	Scenario list; each scenario owns its steps.
`scenario_routes`	Cross-scenario routing rules.
`fact_schema`	Typed fact definitions available to guards and execution.
`analysis_schema`	Structured analysis extraction contract.
`agent_capability_manifest`	Capabilities exposed by the agent.
`metadata`	Authoring and runtime metadata.

Scenario Model

Field	Meaning
`id`, `name`	Stable scenario identity and display name.
`start_step_id`	Initial step for this scenario.
`steps`	Step definitions inside the scenario.
`summary`, `order`	Authoring metadata and display order.
`entry_channels`	Channel constraints for entry.
`resources`	Scenario-scoped runtime resources.
`flow_layout`	Canvas layout data for authoring.

Step Model

Field Group	Fields	Role
Identity	`id`, `name`, `description`	Stable identity and author-facing description.
Conversation behavior	`say`, `response_policy`, `handoff`, `completion`	What the agent says, how it responds, when it hands off, and how the step completes.
Control flow	`transitions`, `guards`, `confidence_threshold`	Next-step selection and condition evaluation.
Facts and tools	`fact_requirements`, `tool_policy`, `action_config`	Data needed before progression and allowed runtime actions.
Execution controls	`workload_class`, `execution_isolation`, `classifier_uses_facts`	Runtime execution and classifier behavior knobs.

Important Correction

The current model uses `start_scenario_id` and scenario-level `start_step_id`. Older references to `entry_scenario_id` or `entry_step_id` are stale.

Knowledge Scope

There is no direct step-level `knowledge_base_ids` field in the current model. Knowledge is resolved through runtime resources and resolver logic.

Authoring Unit

The UI should treat the full document as the persisted edit unit, even when the canvas presents scenarios, steps, and transitions visually.

04 AI, Classifier, And Dialogue

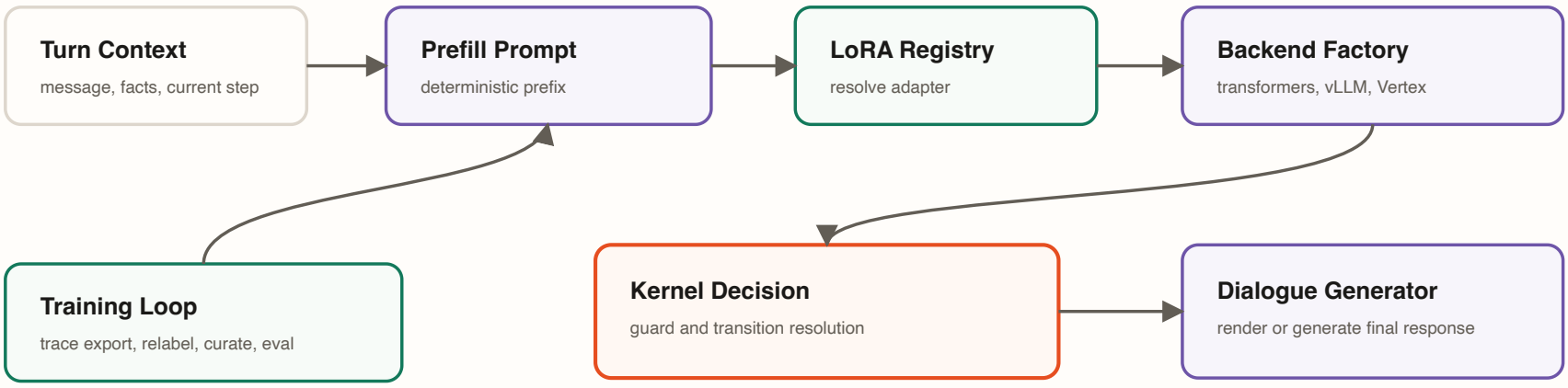
Two-LLM Split

The runtime separates intent/classification from dialogue rendering. The prefill-first classifier chooses structured intent outcomes. The dialogue generator renders or produces user-facing language after deterministic runtime decisions have been made.

Plane	Primary Modules	Backends
Classifier	`classifier/prompt`, `protocol`, `dispatcher`, `factory`	`transformers`, `vllm`, `vertex_gemini`
Dialogue	`response_generation.py`	Gemini / Google / Vertex, default model `gemini-3-flash-preview`
Atlas generation	`atlas_model_gateway.py`, docs/parser paths	Anthropic Claude Sonnet path used by Atlas, not the live dialogue backend

Classifier Lifecycle

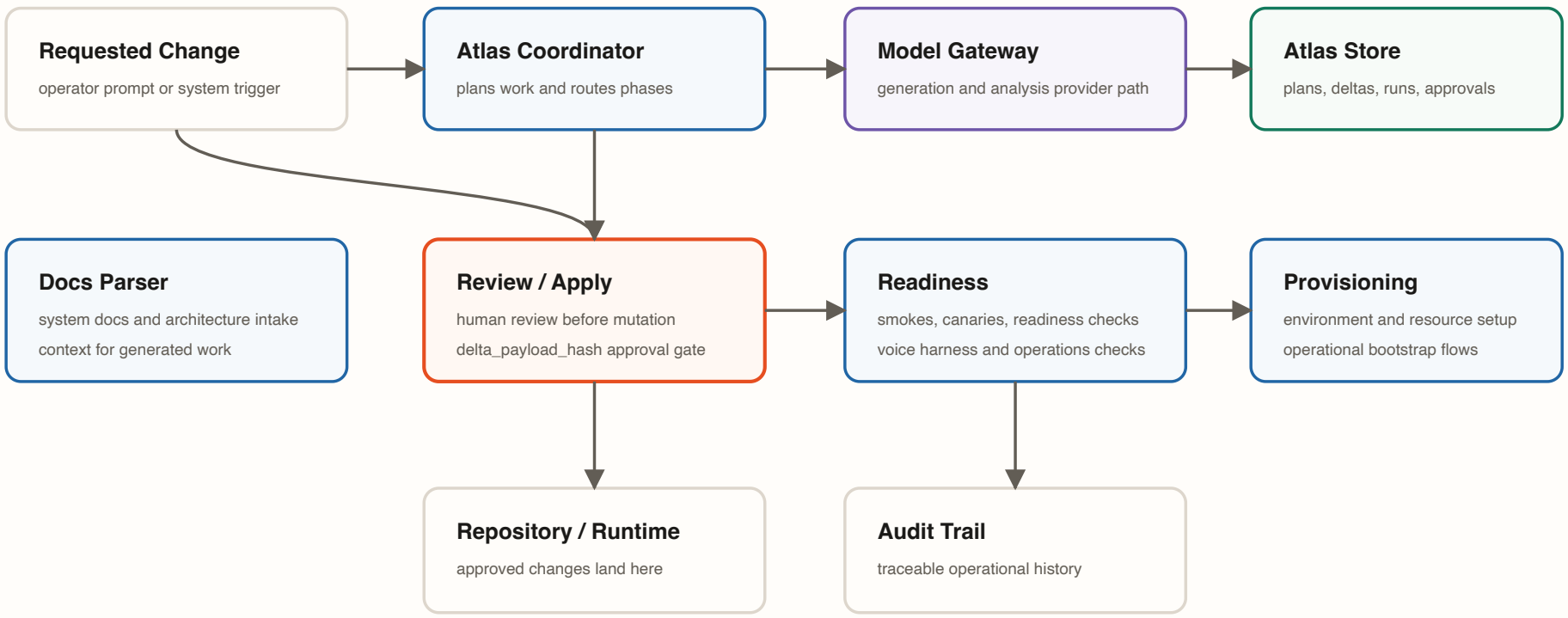
- `prompt` builds deterministic byte-identical classifier prefixes.
- `protocol` defines `PrefillClassifier`, `ClassificationRequest`, and `ClassificationResult`.
- `dispatcher` is the single path from prompt and LoRA resolution to backend invocation.
- `registry` manages LoRA lifecycle and resolution.
- `promotion` and `promotion_api` gate evaluation and status flips.
- `hot_load` and `hot_load_api` support vLLM adapter loading.
- `training_scheduler` and `training_api` manage retrain triggers.



Classifier, registry, and dialogue generation are distinct planes. This is important for evaluation, debugging, and enterprise reliability.

Provider reality. OpenAI appears in voice plugin, sentiment, and tests, but the main live dialogue path in the current codebase is Gemini-oriented. Anthropic Claude Sonnet is used by Atlas paths rather than the live customer conversation generator.

05 Atlas Architecture



Atlas is an operational automation plane. It should be described as review/apply plus readiness, docs, provisioning, and operational workflows.

Current Atlas Module Inventory

Module Family	Examples	Purpose
API and protocol	`atlas_api.py`, `atlas_protocol.py`	HTTP/API boundary and structured protocol objects for Atlas operations.
Coordination and storage	`atlas_coordinator.py`, `atlas_store.py`	Run coordination, persisted plans, deltas, approvals, and operational state.
Generation and context	`atlas_model_gateway.py`, `atlas_docs_parser.py`	Provider access and repository/document context extraction.
Readiness and voice	`atlas_readiness*`, `atlas_voice_harness.py`	Readiness checks, canary-style validation, and voice-oriented verification.
Provisioning and runners	`atlas_provisioning.py`, `atlas_adk_runner.py`	Environment setup and execution support.

Stale reference removed. The current repository does not contain `atlas_apply.py`. Apply/review behavior should be documented through the actual Atlas API, coordinator, store, and approval hashing path.

06 Frontend Application

Stack

Layer	Current Technology
Framework	React `^18.2.0`
Language	TypeScript `^5.9.3`
Build	Vite `^8.0.14`
Routing	React Router DOM 6.x
Server state	TanStack React Query
Client state	Zustand
Styling	Tailwind CSS, Radix UI, lucide-react
Realtime voice	`livekit-client` package lock resolves to 2.16.1

Frontend Shape

- `frontend/src/main.tsx` boots the app.
- `frontend/src/App.tsx` defines route groups, auth gates, and lazy pages.
- `frontend/src/features/agent-canvas` contains the core authoring surface.
- `frontend/src/api/client.ts` provides bearer token, CSRF, and request behavior.
- Feature folders are self-contained; shared UI lives under `components`.

Current Route Inventory

Group	Routes
Public	`/login`, `/signup`, `/register` redirect, `/forgot-password` redirect, `/auth/callback`, `/auth/magic-link`, `/terms`, `/privacy`, `/accept-invitation`, `/confirm-action`, `/integrations/oauth/callback`
Core protected	`/dashboard`, `/agents`, `/agents/:id/canvas`, `/agents/:id/setup`, `/agents/:id/widget`, `/agents/:id/analysis`, `/calls`
Operations	`/browser-tasks`, `/operations/phone-numbers`, `/knowledge-base`, `/tickets`, `/analytics`, `/insights`, `/audit`
Workflow and evaluation	`/intents-tags`, `/kpi-goals`, `/kpi-goals/:goalId`, `/journeys`, `/journeys/:journeyId`, `/rules`, `/tools`, `/evaluation`, `/testing` redirect
Settings and admin	`/settings`, `/settings/:tab`, `/pricing`, `/settings/billing`, `/templates`, `/staff`, `/staff/:section`, `/` redirect to dashboard

07 Data, Templates, And Integration Surface

Persistence

- SQLAlchemy 2.x and Psycopg 3.x are the production database stack.
- Store implementations cover conversations, traces, agents, audit, and related operational records.
- Alembic migrations are active and currently number 91 migration files.
- In-memory stores remain useful for development and tests.

External Integrations

- Voice and realtime: LiveKit SDK, LiveKit Agents, voice plugins.
- Telephony: Telnyx and Africa's Talking provider paths.
- Ticketing: Zendesk, Jira, Freshdesk, Linear.
- Billing: Stripe subscription and usage management.
- Auth: magic link, OAuth, SSO, JWT runtime auth context.

System Templates

Template File	Likely Wedge
`ecommerce-returns-refunds.json`	Commerce support, returns, refunds, order operations.
`healthcare-triage-scheduling.json`	Healthcare intake, triage, booking, care routing.
`isp-customer-support.json`	Connectivity support and technical troubleshooting.
`melonpay-support-demo.json`	Fintech support and account/payment operations.
`nafmfb-financial-support-assistant-demo.json`	Financial institution support, loan/account workflows.
`sales-agent.json`	Lead qualification and revenue workflow automation.
`telecom-billing-disputes-payment-plans.json`	Telecom billing, disputes, collections, and payment plans.

General Platform

Agent documents, rules, tools, facts, journeys, and channels can model many enterprise workflows beyond customer service.

Regional Integration Advantage

Regional value should come from carriers, payment workflows, compliance requirements, local templates, and operational relationships.

Enterprise Wedge

Sell outcomes that move money or reduce operational load: onboarding, collections, sales follow-up, claims, field service, and compliance workflows.

08 Deployment And Testing

Competition Deployment Path

Area	Role	Status
Container image	`infra/docker/Dockerfile` packages the backend runtime for deployment.	**Present**
Google Cloud	`infra/gcp` provides the Google-oriented infrastructure path for challenge hosting.	**Present**
Google AI services	Gemini, ADK, Google Speech-to-Text, and Google Text-to-Speech power the Atlas readiness loop.	**Present**
Evidence storage	Conversation traces, readiness reports, provider metadata, and audit records are persisted for review.	**Present**

Test And Smoke Surface

- Backend: `make test` and direct `pytest` paths.
- Frontend: `npm run build`, `npm run lint`, `npm run test`, Playwright e2e.
- Realtime smoke: widget chat, widget voice, and LiveKit smoke targets.
- Auth UI/browser e2e and ticketing verification targets exist in the project command set.
- Atlas readiness smoke exists as a dedicated command target.

Competition architecture summary. Ruhu combines deterministic runtime control, typed workflow authoring, channel adapters, tools, traces, and Atlas readiness evaluation so enterprises can deploy customer-facing agents with auditable evidence.